PukSquash Developer Manual

Team Practice Bookings

Firebase Data Connect edition | Source revision fe3e361 | 4 October 2026

1 Functionality Overview

Team practice lets an ACTIVE COACH select several court/date/time-slot cells and reserve them for squad training. Each selected cell becomes a Booking with bookingType TEAM_PRACTICE. The operation does not overwrite another reservation.

The team-practice page requests a dedicated availability grid and builds a selections array. Laravel validates the complete selection list, including duplicate cells, dates, active courts and slots and existing reservations. It prepares one booking row per cell and sends them through createMany. Successful booking IDs are returned to the page and can be retrieved for the coach’s selected date.

1 1 Files and Responsibilities

resources/js/pages/CoachTeamPracticeBookingPage.vue — multiple selection interface

resources/js/features/booking/bookingService.js — team-practice requests

app/Http/Controllers/Api/TeamPracticeController.php — availability, store and mine

app/Repositories/BookingRepository.php — createMany and team-practice query

1 2 API Contract

GET /api/coach/team-practice/availability?date=YYYY-MM-DD. POST /api/coach/team-practices accepts {selections:[{court_id,time_slot_id,booking_date}]}. GET /api/coach/team-practices?date=YYYY-MM-DD reads the coach’s sessions.

Reading this part

Code excerpts are from the stated Firebase revision unless explicitly labelled as a proposed extension. Requirements and maintenance gaps are separated from implemented behaviour. Each part can be read independently; Part 00 covers the shared environment.

2 Preparing the selected reservations

app/Http/Controllers/Api/TeamPracticeController.php | source line 283 onward; blank and comment lines omitted

```
$bookedAt = now()->toIso8601ZuluString();
$rows = array_map(fn (array $sel) => [
    'courtId' => $sel['court_id'],
    'timeSlotId' => $sel['time_slot_id'],
    'createdById' => $user->id,
    'coachId' => null,
    'bookingDate' => $sel['booking_date'],
    'bookingType' => 'TEAM_PRACTICE',
    'bookedAt' => $bookedAt,
], $validated['selections']);
$results = $this->bookings->createMany($rows);
```

Explanation

After all selections pass validation, the controller maps REST field names to the GraphQL variables used for ordinary booking insertion. The same timestamp groups the submitted cells operationally, but the database stores a separate Booking for each one.

Physical Code Explanation

1 bookedAt is created once for the submission.

2 Each row contains the court, slot, booking date and the authenticated coach as createdById.

3 coachId is null because this is a team practice rather than a private lesson assigned through the coach relation.

4 bookingType is TEAM_PRACTICE. createMany sends the prepared rows to the repository.

The backend checks a fourteen-day window, rejects past or started cells, verifies active slots and courts, and rejects duplicate or occupied selections. The team route does not apply ordinary per-user overlapping or count checks, which supports multiple courts at the same time.

3 Batch insertion through Data Connect

app/Repositories/BookingRepository.php | source line 74 onward; blank and comment lines omitted

```
public function createMany(array $rows): array
{
    return $this->client->executeMany(
        'CreateBooking',
        array_map(
            fn (array $row) => [
                'courtId'     => $row['courtId'],
                'timeSlotId'  => $row['timeSlotId'],
                'createdById' => $row['createdById'],
                'coachId'     => $row['coachId'] ?? null,
                'bookingDate' => $row['bookingDate'],
                'bookingType' => $row['bookingType'],
                'bookedAt'    => $row['bookedAt'],
            ],
            $rows,
        ),
    );
}
```

Explanation

The repository maps each row and asks SqlConnectClient::executeMany to issue multiple CreateBooking requests. The client returns an ok flag and either data or an error for each request. This is parallel request execution, not a single PostgreSQL transaction.

Physical Code Explanation

1 The operation name remains CreateBooking, so the same core insertion contract is reused.

2 array_map ensures each request has exactly the required GraphQL variables. coachId falls back to null.

3 executeMany preserves result order for the caller to examine and returns successes and failures separately.

The controller attempts to cancel IDs already accumulated when it encounters a failure. This compensation can itself fail, and later requests may already have succeeded. A failed batch can therefore leave residual reservations. Replace this with a verified atomic operation before promising all-or-nothing booking.

4 Verification and Maintenance

Verification checklist

1 Book several free cells across courts and inspect the returned booking_ids and TEAM_PRACTICE rows.

2 Submit an occupied cell, inactive court, inactive slot, duplicate cell, past date and date beyond fourteen days; verify the specified 409 or 422 rejection.

3 Call the endpoints as a non-coach or inactive coach: expect 403.

4 Inject failure into the middle of a batch and inspect every selected cell, including later successes. Reconcile residual bookings rather than assuming rollback.

5 Run TeamPracticeBookingTest, then add real connector tests for concurrent batches and compensation failures.

Implementation boundaries

TEAM_PRACTICE is used by the implementation even though older schema comments enumerate only the three ordinary booking types. The field is a string, so documentation and validation must be maintained together.

This baseline does not create ordinary Player 1 rows for team-practice insertions. The creator identifies the responsible coach; avoid interpreting an empty participant list as a free court.

4 1 Source References

PukSquash repository, revision fe3e361; source paths are identified beside each excerpt. Requirements: Functional Specifications PukSquash (3) and Techinical specifications PukSquash (2). Business context: Business_Case and Feasibility_Study for PukSquash. Structure reference: Campus Marketplace Developer Manual (4).